

Strings em Java

Manipulação e Comparação

Universidade Estadual da Paraíba
Linguagem de Programação II

September 22, 2026

O que vamos aprender hoje?

- ① O que é uma **String**, de verdade
- ② Como **ler** strings digitadas pelo usuário
- ③ Por que strings são **imutáveis**
- ④ Como **manipular** strings (métodos prontos, percorrer, virar array)
- ⑤ Como **comparar** strings corretamente
- ⑥ Exemplos práticos: **inverter** uma string

Sumário

- 1 O que é uma String
- 2 Lendo Strings
- 3 Strings São Imutáveis
- 4 Manipulando Strings
- 5 Comparando Strings
- 6 Exemplos Práticos: Juntando Tudo

Sumário

- 1 O que é uma String
- 2 Lendo Strings
- 3 Strings São Imutáveis
- 4 Manipulando Strings
- 5 Comparando Strings
- 6 Exemplos Práticos: Juntando Tudo

Precisamos representar texto

- Nomes, frases, mensagens, endereços...
- Quase todo programa lida com texto em algum momento.

Em Java, quem representa texto é a **String**.

Relembrando os Tipos Primitivos

Tipo	Guarda
<code>int</code>	números inteiros
<code>double</code>	números com casas decimais
<code>boolean</code>	true ou false
<code>char</code>	um único caractere

O que é uma String?

😊 Definição: String

É uma sequência de caracteres, usada para representar **texto** – uma palavra, uma frase, ou qualquer conjunto de letras, números e símbolos.

String NÃO é um tipo primitivo!

Mesmo sendo usada o tempo todo como se fosse "simples", String é uma **classe** – ou seja, toda String que criamos é um **objeto**.

Uma Dica para Reconhecer

Primitivos	Classes (objetos)
começam com letra minúscula int, double, boolean, char	começam com letra Maiúscula String, Scanner, ...

Dica

Em Java, por convenção, nomes de classes começam com letra maiúscula. Reparar nisso já ajuda a identificar o que é objeto!

String é uma Classe

😊 Definição: Classe String

`String` é uma classe pronta do Java (`java.lang.String`). Toda vez que escrevemos um texto entre aspas, estamos criando um **objeto** dessa classe.

Primitivo x Objeto

Tipo Primitivo

Representa um **valor simples**.

```
1 int idade = 20;
```

20

Exemplos: int, double, boolean, char.

Objeto

É criado a partir de uma **classe** e pode possuir métodos.

```
1 String nome = "Maria";
```

Podemos realizar operações usando métodos:

```
1 nome.length();  
2 nome.toUpperCase();
```

Primitivo x Objeto: Resumindo

Tipo Primitivo

- Guarda um **valor simples**;
- Exemplos:
 - int
 - double
 - char
 - boolean
- Não possui métodos próprios.

Objeto

- É criado a partir de uma **classe**;
- Exemplos:
 - String
 - Scanner
- Pode possuir métodos.

♥ Checkpoint

String não é um tipo primitivo: ela é uma **classe**, e cada String criada é um **objeto**.

String: uma analogia

Analogia

O Java trata String com "cuidados especiais": permite criar com aspas, juntar com +, imprimir direto... Isso é só **açúcar sintático** (uma forma mais fácil de escrever) – por trás das cortinas, é um objeto de verdade, como qualquer outro.

Criando uma String: Literal

```
1 String nome = "Maria";
```

- Essa é a forma **mais comum** de criar uma String;
- O texto entre aspas já cria o objeto – não precisamos escrever `new`.

Criando uma String: `new String()`

```
String nome = new String("Maria");
```

- Também é possível criar com `new`, como qualquer outro objeto;
- Na prática, **quase ninguém** escreve assim – a forma literal é preferida.

♡ Checkpoint

- String representa texto, mas **não** é primitivo;
- É uma **classe** – toda String é um objeto;
- O Java dá "facilidades" (aspas, +) para trabalhar com ela, mas por dentro continua sendo um objeto de verdade.

Sumário

- 1 O que é uma String
- 2 Lendo Strings
- 3 Strings São Imutáveis
- 4 Manipulando Strings
- 5 Comparando Strings
- 6 Exemplos Práticos: Juntando Tudo

Relembrando o Scanner

- Já vimos que o `Scanner` lê dados digitados pelo usuário;
- Para ler texto, usamos dois métodos: `next()` e `nextLine()`.

next() x nextLine()

Método	O que lê
<code>next()</code>	uma palavra só (para no espaço)
<code>nextLine()</code>	a linha inteira (inclui espaços)

😊 Dica

Para ler um nome completo ("Maria Silva"), use `nextLine()` – o `next()` pararia em "Maria".

Exemplo Prático

😊 Exemplo: Lendo o nome completo

```
1 Scanner leitor = new Scanner(System.in);  
2 System.out.print("Digite seu nome completo: ");  
3 String nome = leitor.nextLine();  
4 System.out.println("Ola, " + nome + "!");
```

Por que o Enter pode causar problema?

Quando digitamos um valor e pressionamos **Enter**, o teclado envia também uma quebra de linha.

Por exemplo, ao digitar:

20 + Enter

o programa recebe algo semelhante a:

20\n

😊 Dica

Alguns métodos do Scanner leem apenas o valor digitado e deixam o **Enter** esperando na entrada.

O Caso Clássico: `nextInt()` e `nextLine()`

```
1 Scanner leitor = new Scanner(System.in);  
2  
3 System.out.print("Digite sua idade: ");  
4 int idade = leitor.nextInt();  
5  
6 System.out.print("Digite seu nome: ");  
7 String nome = leitor.nextLine();  
8  
9 System.out.println("Nome: " + nome);
```

Erro Comum

O `nextInt()` lê o número, mas não consome o Enter. Logo depois, o `nextLine()` encontra esse Enter e entende que a linha já terminou.

Como Resolver?

Basta consumir o Enter antes de ler a próxima linha:

```
1 Scanner leitor = new Scanner(System.in);
2
3 System.out.print("Digite sua idade: ");
4 int idade = leitor.nextInt();
5
6 leitor.nextLine();
7
8 System.out.print("Digite seu nome: ");
9 String nome = leitor.nextLine();
10
11 System.out.println("Nome: " + nome);
```

Dica

Neste caso, o primeiro `nextLine()` serve apenas para consumir o Enter que ficou na entrada.

Quando Precisamos Ter Cuidado?

Sequência	Precisa de cuidado?
<code>nextInt() → nextInt()</code>	Não
<code>next() → next()</code>	Não
<code>nextLine() → nextLine()</code>	Não
<code>nextInt() → nextLine()</code>	Sim
<code>nextDouble() → nextLine()</code>	Sim
<code>next() → nextLine()</code>	Sim

♡ Checkpoint

Se usamos `nextLine()` depois de métodos como `nextInt()`, `nextDouble()` ou `next()`, devemos lembrar do Enter que pode ter ficado na entrada.

♡ Checkpoint

- `next()`: lê uma palavra;
- `nextLine()`: lê a linha inteira, com espaços;
- Lembre-se do cuidado com `nextInt()` seguido de `nextLine()` – o Enter sobrando!

Sumário

- 1 O que é uma String
- 2 Lendo Strings
- 3 Strings São Imutáveis**
- 4 Manipulando Strings
- 5 Comparando Strings
- 6 Exemplos Práticos: Juntando Tudo

O que significa "imutável"?

😊 Definição: Imutabilidade

Uma vez criada, o **conteúdo** de uma String **nunca** muda. Nenhum método "edita" a string original – todos criam e devolvem uma String **nova**.

Imutabilidade: uma analogia

Analogia

Pense numa placa **gravada em pedra**. Depois de gravada, você não consegue apagar ou editar uma letra – se quiser um texto diferente, precisa esculpir uma placa **nova**. A placa antiga continua lá, intacta.

Vamos Testar

```
1 String nome = "maria";  
2 String maiuscula = nome.toUpperCase();  
3  
4 System.out.println(nome);           // maria  
5 System.out.println(maiuscula);      // MARIA
```

Repare: nome continua minúsculo! O `toUpperCase()` não mudou a string original.

O que Aconteceu de Verdade

- `nome.toUpperCase()` **não** alterou o texto "maria";
- Ele criou um texto **novo**, "MARIA", e **devolveu** esse novo texto;
- Guardamos esse novo texto numa variável diferente: `maiuscula`.

Visualizando a Imutabilidade



A caixa nome nunca foi tocada – uma caixa nova, maiuscula, foi criada do lado.

Imutabilidade de Strings

```
1 String s1 = "abc";  
2 String s2 = s1;  
3  
4 System.out.println(s1); // abc  
5 System.out.println(s2); // abc  
6  
7 s1 = s1.toUpperCase();  
8  
9 System.out.println(s1); // ABC  
0 System.out.println(s2); // abc
```

Antes do toUpperCase()

s1 "abc" s2 "abc"

Depois de s1 = s1.toUpperCase()

s1 "ABC" s2 "abc"

😊 Dica

O método `toUpperCase()` não altera a String original.
Ele cria uma **nova String** com o valor "ABC" e `s1` passa a usar esse novo valor.
A variável `s2` continua com "abc".

😊 Dica

Como nenhuma String muda sozinha, **sempre guarde o retorno** do método numa variável (a mesma ou outra). Se você só chamar `nome.toUpperCase()`; e não guardar o resultado em lugar nenhum, o novo objeto criado se perde!

☹ Erro Comum

Achar que um método como `toUpperCase()` **altera** a string original. Ele nunca faz isso – sempre devolve uma `String` nova. Se você não guardar esse retorno, é como se nada tivesse acontecido.

Mas... se eu posso fazer `saudacao += "!"`, igualzinho eu faço com `int`, isso não significa que `String` virou um tipo primitivo?

Não! Mesmo usando `+=`, `String` continua sendo um objeto – e continua imutável.

O `+=` é só mais um "açúcar sintático". Vamos ver o que ele faz de verdade.

O que o Java Faz de Verdade

```
1 String saudacao = "Ola";  
2 saudacao += ", mundo!";
```

Por trás dos panos, o Java entende a segunda linha assim:

```
1 saudacao = saudacao + ", mundo!";
```

Passo a Passo do += em String

- ❶ O Java pega o texto atual de saudacao ("Ola") e junta com ", mundo!";
- ❷ Isso cria uma String **totalmente nova**: "Ola, mundo!";
- ❸ A variável saudacao é **reatribuída** – passa a guardar esse texto novo;
- ❹ O texto antigo ("Ola") deixou de ser usado – ele não existe mais em lugar nenhum do programa.

int += x String +=

total

int (primitivo)

5 → 8

O valor de dentro da caixa é alterado.

saudacao

String (objeto)

"Ola" → "Ola, mundo!"

Por fora parece igual – mas por dentro, o texto antigo foi descartado e um texto novo foi criado.

☹ Erro Comum

Achar que += "edita" a string original por dentro. Na verdade, ele sempre cria uma String nova e **reatribui** a variável – o mecanismo é o mesmo do toUpperCase() e de qualquer outro método de String.

♡ Checkpoint

- Nenhum método de String muda a string original – todos criam uma **nova**;
- += não é exceção: por trás dos panos, é `saudacao = saudacao + "..."`;
- String continua sendo um **objeto imutável**, mesmo com essa sintaxe parecida com a de primitivos.

Sumário

- 1 O que é uma String
- 2 Lendo Strings
- 3 Strings São Imutáveis
- 4 Manipulando Strings**
- 5 Comparando Strings
- 6 Exemplos Práticos: Juntando Tudo

- Como `String` é uma classe, ela vem com vários **métodos** prontos;
- Cada um devolve uma informação, ou uma `String` nova – nunca altera a original.

length()

```
1 nome.length()
```

Devolve a **quantidade de caracteres** da String.

Exemplo: length()

☺ Exemplo: Tamanho do nome

```
1 String nome = "Maria";  
2 System.out.println(nome.length()); // 5
```

charAt(indice)

```
nome.charAt(indice)
```

Devolve o **caractere** numa posição específica. Assim como em vetores, o **índice começa em 0**.

Exemplo: charAt()

☺ Exemplo: Primeira letra

```
1 String nome = "Maria";  
2 System.out.println(nome.charAt(0)); // M
```

substring(inicio, fim)

```
1 frase.substring(inicio, fim)
```

Devolve um **pedaço** da String, do índice `inicio` até o `fim` – **sem incluir** o `fim`.

Exemplo: substring()

😊 Exemplo: Pegando "Java"

```
1 String frase = "Java e incrível";  
2 System.out.println(frase.substring(0, 4)); // Java
```

😞 Erro Comum

O índice 4 **não** entra no resultado – por isso pegamos exatamente as letras dos índices 0, 1, 2 e 3.

indexOf(texto)

```
1 frase.indexOf(texto)
```

Devolve o **índice** onde texto começa a aparecer – ou -1 se não encontrar.

Exemplo: indexOf()

☺ Exemplo: Onde está "incrível"

```
1 String frase = "Java e incrível";  
2 System.out.println(frase.indexOf("incrível")); // 7
```

toUpperCase() e toLowerCase()

```
1 nome.toUpperCase()  
2 nome.toLowerCase()
```

Devolvem uma String nova, toda em **maiúsculas** ou **minúsculas**.

Exemplo: toUpperCase() / toLowerCase()

☺ Exemplo: Mudando a caixa das letras

```
1 String nome = "Maria";  
2 System.out.println(nome.toUpperCase()); // MARIA  
3 System.out.println(nome.toLowerCase()); // maria
```

trim()

```
1 nome.trim()
```

Devolve a String **sem espaços** no início e no fim (o meio não é afetado).

☺ Exemplo: Limpando espaços

```
1 String nome = "    Maria    ";  
2 System.out.println(nome.trim()); // "Maria"
```

replace(antigo, novo)

```
1 frase.replace(antigo, novo)
```

Devolve uma String nova, trocando todas as ocorrências de antigo por novo.

☺ Exemplo: Trocando uma palavra

```
1 String frase = "Java e dificil";  
2 System.out.println(frase.replace("dificil", "facil"));
```

contains(texto)

```
1 frase.contains(texto)
```

Devolve true se texto aparece em algum lugar da String.

☺ Exemplo: Verificando se contem

```
1 String frase = "Java e incrivel";  
2 System.out.println(frase.contains("incrivel")); // true
```


split(separador)

```
1 frase.split(separador)
```

Quebra a String em pedaços, usando separador como corte – e devolve um **vetor de Strings**!

Exemplo: split()

😊 Exemplo: Separando frutas

```
1 String frase = "maca,banana,uva";  
2 String[] frutas = frase.split(",");  
3 System.out.println(frutas[0]); // maca  
4 System.out.println(frutas[1]); // banana
```

😊 Dica

Lembra dos vetores? `split()` devolve exatamente isso – e o índice também começa em 0!

Percorrendo uma String

- Às vezes precisamos olhar **cada letra** de uma String, uma por uma;
- Fazemos isso com um `for`, igual percorremos um vetor – usando `length()` e `charAt()`.

Exemplo Prático

😊 Exemplo: Imprimindo letra por letra

```
1 String palavra = "casa";  
2 for (int i = 0; i < palavra.length(); i++) {  
3     System.out.println(palavra.charAt(i));  
4 }
```

😊 Dica

Esse padrão – for de 0 até length(), chamando charAt(i) – vai aparecer em quase todo problema que manipula String letra a letra.

De String para Array: toCharArray()

```
1 char[] letras = palavra.toCharArray();
```

- Transforma a String num **vetor de caracteres** (char[]);
- Depois de virar vetor, podemos percorrê-lo com for e .length (sem parênteses, porque agora é vetor!).

😊 Exemplo: String para array e de volta

```
1 String palavra = "casa";  
2 char[] letras = palavra.toCharArray();  
3  
4 for (int i = 0; i < letras.length; i++) {  
5     System.out.println(letras[i]);  
6 }
```

De Array para String

```
1 String nova = new String(letras);  
2 // ou  
3 String nova = String.valueOf(letras);
```

- As duas formas fazem a mesma coisa: juntam o vetor de char de volta numa String;
- Útil depois de modificar as letras dentro de um vetor, por exemplo.

♡ Checkpoint

- Para percorrer uma String, use for com `length()` e `charAt(i)`;
- `toCharArray()` transforma a String num vetor de char;
- `new String(vetor)` ou `String.valueOf(vetor)` voltam de vetor para String.

equalsIgnoreCase(texto)

```
1 nome.equalsIgnoreCase(texto)
```

Compara o **conteúdo**, mas ignora se as letras são maiúsculas ou minúsculas.

☺ Exemplo: Comparando sem diferenciar caixa

```
1 String resposta = "SIM";  
2 System.out.println(resposta.equalsIgnoreCase("sim")); // true
```

isEmpty()

```
1 nome.isEmpty()
```

Devolve true se a String **não tem nenhum caractere** (tamanho zero).

😊 Exemplo: Verificando se ficou vazia

```
1 String texto = "";  
2 System.out.println(texto.isEmpty()); // true
```

😊 Dica

É comum usar `.trim().isEmpty()` juntos, para tratar também textos com só espaços em branco.

startsWith() e endsWith()

```
1 nome.startsWith(texto)
2 nome.endsWith(texto)
```

Verificam se a String **começa** ou **termina** com um determinado texto.

😊 Exemplo: Verificando início e fim

```
1 String arquivo = "relatorio.pdf";
2 System.out.println(arquivo.endsWith(".pdf")); // true
```

Um Problema Muito Comum

- O Scanner sempre lê como `String`, mesmo quando digitamos um número;
- E às vezes temos um número, mas precisamos dele como texto;
- Para isso, existem métodos prontos de **conversão**.

Convertendo String em Número

```
1 Integer.parseInt(texto)
2 Double.parseDouble(texto)
```

Transformam uma String que contém dígitos num número de verdade, pronto para contas.

😊 Exemplo: De texto para numero

```
1 String texto = "25";
2 int idade = Integer.parseInt(texto);
3 System.out.println(idade + 5); // 30
```

Erro Comum

```
1 String texto = "vinte e cinco";  
2 int idade = Integer.parseInt(texto); // erro!
```

☹ Erro Comum

NumberFormatException: acontece quando o texto **não** representa um número válido. `parseInt` só aceita dígitos (com sinal opcional), nunca palavras.

Convertendo Número em String

```
1 String.valueOf(numero)
```

Transforma um número (ou outro valor) em texto.

😊 Exemplo: De numero para texto

```
1 int idade = 20;  
2 String texto = String.valueOf(idade);  
3 System.out.println("Idade: " + texto);
```

😊 Dica

Concatenar com + (como "Idade: " + idade) já faz essa conversão sozinho – `String.valueOf()` é útil quando você precisa da String separada.

Concatenação com +

```
1 String nome = "Maria";  
2 int idade = 20;  
3 System.out.println("Nome: " + nome + ", idade: " + idade);
```

O + junta textos (e converte números automaticamente para texto).

♡ Checkpoint

- `length()`, `charAt()`, `substring()`, `indexOf()`;
- `toUpperCase()`, `toLowerCase()`, `trim()`, `replace()`, `contains()`;
- `split()` devolve um **vetor** de Strings;
- `equalsIgnoreCase()`, `isEmpty()`, `startsWith()`, `endsWith()`;
- `Integer.parseInt()` / `Double.parseDouble()` convertem texto em número; `String.valueOf()` faz o caminho inverso;
- Nenhum deles muda a String original – todos devolvem algo novo.

Sumário

- 1 O que é uma String
- 2 Lendo Strings
- 3 Strings São Imutáveis
- 4 Manipulando Strings
- 5 Comparando Strings**
- 6 Exemplos Práticos: Juntando Tudo

Como Comparar Duas Strings?

- Com números, comparamos com `==` sem problema: `idade == 18;`
- Será que funciona igual com `String`?

Tentativa com ==

```
1 String a = "Java";  
2 String b = "Java";  
3 System.out.println(a == b); // true
```

Parece funcionar... mas é uma **armadilha!**

Quando == "engana"

```
1 String a = new String("Java");  
2 String b = new String("Java");  
3 System.out.println(a == b); // false!
```

Mesmo conteúdo ("Java"), mas == diz que são **diferentes!**

Por que isso Acontece

- String é um **objeto**, não um número ou um boolean;
- Para objetos, `==` verifica se são **exatamente a mesma coisa criada**, não se o **conteúdo** é igual;
- Com `new String(...)`, criamos duas Strings **diferentes**, mesmo com o mesmo texto dentro.

Visualizando o Problema



Mesmo conteúdo, mas são **duas caixas diferentes** – por isso `==` devolve `false`.

A Forma Correta: equals()

```
1 String a = new String("Java");  
2 String b = new String("Java");  
3 System.out.println(a.equals(b)); // true
```

- .equals() compara o **conteúdo** das Strings, e não se são "a mesma caixinha";
- Funciona **sempre** corretamente, não importa como as Strings foram criadas.

😊 Dica

Sempre use `.equals()` para comparar o conteúdo de Strings. Nunca confie no `==` – mesmo que ele "pareça" funcionar às vezes (por causa de uma otimização interna do Java com textos literais).

☹ Erro Comum

Usar `==` para comparar Strings, como `if (nome == "Maria")`. Pode até funcionar por acidente em alguns casos, mas não é confiável – use sempre `if (nome.equals("Maria"))`.

♡ Checkpoint

- `==` compara se é **a mesma String criada** – não use para comparar texto;
- `.equals()` compara **conteúdo** – essa é a forma correta;
- Isso vale para qualquer objeto em Java, não só String.

Sumário

- 1 O que é uma String
- 2 Lendo Strings
- 3 Strings São Imutáveis
- 4 Manipulando Strings
- 5 Comparando Strings
- 6 Exemplos Práticos: Juntando Tudo**

Agora vamos usar o que aprendemos – `charAt`, `length`, `substring`, `isEmpty`, `contains` – para resolver dois problemas clássicos.

Lembrete Rápido: Recursão

😊 Definição: Recursão

Uma função **recursiva** é aquela que chama a **si mesma**, resolvendo um pedaço menor do problema a cada chamada.

Toda função recursiva precisa de:

- 1 Um **caso base**: quando parar de chamar a si mesma;
- 2 Um **caso recursivo**: chama a si mesma com um problema **menor**.

Problema 1: Inverter uma String

Dada uma palavra, como escrever ela **de trás para frente**?

"casa" → "asac"

Vamos ver **duas** formas de resolver: iterativa e recursiva.

Versão Iterativa: a Ideia

- Começamos com uma String vazia invertida;
- Percorremos a palavra original de **trás para frente**;
- A cada letra, vamos **concatenando** (`+=`) na String invertida.

Versão Iterativa: o Código

😊 Exemplo: Invertendo com for

```
1 String palavra = "casa";
2 String invertida = "";
3 for (int i = palavra.length() - 1; i >= 0; i--) {
4     invertida += palavra.charAt(i);
5 }
6 System.out.println(invertida); // asac
```

😊 Dica

Repare: o for começa em `length() - 1` (o último índice válido) e vai **diminuindo** até 0.

Versão Recursiva: a Ideia

- **Caso base:** se a String está vazia, o "inverso" dela é ela mesma (vazia);
- **Caso recursivo:** o inverso de uma palavra é o **inverso do restante** (sem a primeira letra), seguido da **primeira letra** no final.

Versão Recursiva: o Código

😊 Exemplo: Invertendo com recursao

```
1 public static String inverterRecursivo(String s) {  
2     if (s.isEmpty()) {  
3         return s;  
4     }  
5     return inverterRecursivo(s.substring(1)) + s.charAt(0);  
6 }
```

- `s.isEmpty()`: nosso caso base;
- `s.substring(1)`: a palavra **sem** a primeira letra – o problema menor;
- `+ s.charAt(0)`: a primeira letra, colocada **no final**.

Desenrolando a Recursão

```
inverterRecursivo("abc") = inverterRecursivo("bc") + 'a'  
                        = (inverterRecursivo("c") + 'b') + 'a'  
                        = ((inverterRecursivo("") + 'c') + 'b') + 'a'  
                        = (" " + 'c') + 'b' + 'a'  
                        = "cba"
```

Cada chamada resolve um pedacinho **menor**, até chegar na String vazia.

♡ Checkpoint

- Iterativa: percorre de trás para frente, concatenando numa String nova;
- Recursiva: caso base (`isEmpty()`) + caso recursivo (`substring(1)` e `charAt(0)`);
- As duas usam só métodos prontos de String que já conhecemos!